CORRIGÉ · PROBLÈME · ANNALES SPÉ

Analyse d'un texte, dictionnaires et SQL

Énoncé

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Alice compresse un message, l'encode, le transmet ; Bob le décode puis le décompresse. On étudie la compression (codage arithmétique) puis le décodage (algorithme de Viterbi).

Avec le code de longueur fixe `'a'` = 00, `'b'` = 01, `'c'` = 10, la chaîne `s='abaabaca'` tient sur 16 bits. On peut coder `'a'` sur 1 bit et `'b'`, `'c'` sur 2 bits pour obtenir 11 bits.

1. Proposer une telle représentation décodable sans ambiguïté, avec code(`'a'`) < code(`'b'`) < code(`'c'`).

2. Écrire `nbCaracteres(c:str, s:str) -> int` (nombre d'occurrences de `c` dans `s`), linéaire en $n =$ `len(s)`.

3. Que renvoie `listeCaracteres('abaabaca')` ? Expliquer son principe.

```
def listeCaracteres(s:str):
    listeCar = []
    n = len(s)
    for i in range(n):
        c = s[i]
        if not(c in listeCar):
            listeCar.append(c)
    return listeCar
```

4. Complexité dans le pire cas en fonction de n et du nombre k de caractères distincts (le test `c in listeCar` est linéaire en la longueur de `listeCar`).

5. Que fait `analyseTexte` ? Valeur de `analyseTexte('babaaaabca')` ?

```
def analyseTexte(s:str):
    R = []
    l = listeCaracteres(s)
    for i in range(len(l)):
        c = l[i]
        R.append((c, nbCaracteres(c, s)))
    return R
```

6. Complexité de `analyseTexte` en fonction de n et k.

7. Réécrire `analyseTexte` avec un dictionnaire, en complexité linéaire en n, indépendante de k, avec un seul parcours de la chaîne. Exemple : `analyseTexte('abracadabra')` renvoie `{'a':5, 'b':2, 'r':2, 'c':1, 'd':1}`.

Base de données : `caractere(idCar, symbole, typeCaractere, codeHTML)` ; `corpus(idLivre, titre, auteur, annee, nombreCaracteres, langue)` ; `occurrences(idCar, idLivre, nombreOccurrences)`

1. Requête SQL : liste sans doublon des auteurs de `corpus`.
2. Requête SQL : pour chaque caractère, le symbole et sa fréquence en `'Français'` (nombre total d'occurrences divisé par le nombre total de caractères des textes du corpus).

Décoder l'énoncé

Ce que l'énoncé te donne.

- Interdit : `count`, `max`… On écrit les boucles soi-même.

Ce qu'on te demande. Compter les caractères d'un texte, d'abord naïvement, puis en un seul parcours avec un dictionnaire ; écrire deux requêtes SQL.

Corrigé

Question 1

Étape 1. Q1. Un code de longueur variable

On propose :

$$\texttt{'a'} \mapsto 0, \qquad \texttt{'b'} \mapsto 10, \qquad \texttt{'c'} \mapsto 11.$$

Nombre de bits : `'abaabaca'` contient 5 fois `'a'`, 2 fois `'b'`, 1 fois `'c'` : $5\times1+2\times2+1\times2=11$ bits : `0 10 0 0 10 0 11 0`.

Pas d'ambiguïté : aucun code n'est le début (préfixe) d'un autre (on dit que le code est **préfixe**). On décode en lisant de gauche à droite : un `0` est forcément un `'a'` ; un `1` annonce `'b'` ou `'c'`, et le bit suivant tranche.

Ordre : en binaire, $0 < 10 < 11$ (lus comme $0, 2, 3$), comme demandé.

Question 2

Étape 1. Q2. Compter les occurrences

```
def nbCaracteres(c, s):
    compteur = 0
    for i in range(len(s)):
        if s[i] == c:          # une occurrence de plus
            compteur = compteur + 1
    return compteur
```

Une boucle de n tours, chaque tour en temps constant : complexité $O(n)$.

Question 3

Étape 1. Q3. La liste des caractères utilisés

On parcourt `'abaabaca'` : `'a'` est ajouté (liste vide), `'b'` est ajouté, les `'a'` et `'b'` suivants sont déjà présents, `'c'` est ajouté.

La fonction renvoie `['a', 'b', 'c']`.

Principe : on parcourt la chaîne et on ajoute chaque caractère à `listeCar` seulement s'il n'y est pas encore. On obtient les caractères distincts, dans l'ordre de leur première apparition.

Question 4

Étape 1. Q4. Complexité de listeCaracteres

La boucle fait n tours. À chaque tour, le test `c in listeCar` coûte au plus la longueur de `listeCar`, qui est toujours $\leqslant k$ (elle ne contient que des caractères distincts de `s`). Le reste du tour est en temps constant.

Complexité dans le pire cas : $O(nk)$.

Ce pire cas est atteint, par exemple quand les k caractères distincts apparaissent au début puis que la suite répète le dernier : chaque test parcourt alors toute la liste.

Question 5

Étape 1. Q5. Que fait analyseTexte ?

Rôle : la fonction renvoie la liste des couples (caractère, nombre d'occurrences), un couple par caractère distinct de `s`, dans l'ordre de première apparition.

Exemple `'babaaaabca'` (positions 0 à 9) : `listeCaracteres` donne `['b', 'a', 'c']`. On compte : `'b'` en positions 0, 2, 7 (3 fois) ; `'a'` en positions 1, 3, 4, 5, 6, 9 (6 fois) ; `'c'` en position 8 (1 fois). Vérification : $3 + 6 + 1 = 10$ caractères.

```
[('b', 3), ('a', 6), ('c', 1)]
```

Question 6

Étape 1. Q6. Complexité de analyseTexte

- Ligne 3 : `listeCaracteres(s)` coûte $O(nk)$ (Q4).
- Boucle : k tours, chacun appelle `nbCaracteres`, en $O(n)$ (Q2) : en tout $O(nk)$.

Complexité : $O(nk) + O(nk) = O(nk)$.

Question 7

Étape 1. Q7. Un seul parcours avec un dictionnaire

```
def analyseTexte(s):
    compte = {}                 # caractère -> nombre d'occurrences
    for i in range(len(s)):     # un seul parcours de s
        c = s[i]
        if c in compte:         # test en temps constant
            compte[c] = compte[c] + 1
        else:
            compte[c] = 1       # première apparition
    return compte
```

Complexité : n tours, chacun en temps constant (test d'appartenance d'une clé et mise à jour d'un dictionnaire en $O(1)$). Total $O(n)$, indépendant de k, et la chaîne n'est parcourue qu'une fois.

Exemple : `analyseTexte('abracadabra')` renvoie `{'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}`.

Question 8

Étape 1. Q8. Les auteurs sans doublon

```
SELECT DISTINCT auteur
FROM corpus;
```

`DISTINCT` retire les doublons : Zola n'apparaît qu'une fois même s'il a plusieurs livres dans le corpus.

Question 9

Étape 1. Q9. Fréquence de chaque caractère en français

Démarche :

- jointure `caractere` – `occurrences` (par `idCar`) pour avoir le symbole, puis `occurrences` – `corpus` (par `idLivre`) pour avoir la langue ;
- on ne garde que les livres en français ;
- on regroupe par caractère et on somme les occurrences ;
- on divise par le nombre total de caractères des livres en français, calculé par une sous-requête.

```
SELECT ca.symbole,
       SUM(o.nombreOccurrences) * 1.0 /
         (SELECT SUM(nombreCaracteres) FROM corpus
          WHERE langue = 'Français') AS frequence
FROM caractere AS ca
JOIN occurrences AS o ON o.idCar = ca.idCar
JOIN corpus AS co ON co.idLivre = o.idLivre
WHERE co.langue = 'Français'
GROUP BY ca.idCar, ca.symbole;
```

Le `* 1.0` force une division décimale (sinon certains systèmes font une division entière, qui donnerait 0). On regroupe par `idCar` (clé primaire) et l'on cite aussi `symbole` pour pouvoir l'afficher. La fréquence est bien entre 0 et 1.

Le piège

En Q9, diviser par `COUNT(*)` ou oublier de restreindre le total aux livres en français : on n'obtiendrait plus une fréquence.

La méthode

Pour compter en un seul parcours : un dictionnaire de compteurs. Pour une complexité : nombre de tours de boucle multiplié par le coût d'un tour, sans oublier les tests d'appartenance cachés.

Rappels de cours utilisés

- **Complexité d'une boucle.** On compte les opérations élémentaires : une boucle de n tours dont le corps coûte $O(c)$ coûte $O(nc)$. Deux boucles imbriquées de n et k tours : $O(nk)$. Un test `x in L` sur une liste coûte $O(\text{len}(L))$; sur un dictionnaire, $O(1)$ en moyenne.
- **Dictionnaires en Python.** `d = {}` crée un dictionnaire vide ; `c in d` teste la présence d'une clé ; `d[c] = v` crée ou modifie ; `d[c] += 1` incrémente une valeur existante.

- **SQL : agrégation et jointure.** `SELECT DISTINCT` retire les doublons. `JOIN ... ON` relie deux tables par une clé. `GROUP BY` regroupe les lignes, et `SUM`, `COUNT` calculent sur chaque groupe. Une sous-requête entre parenthèses peut fournir une valeur.